



Taxonomía de los Lenguajes de Programación

Paradigmas y Administración de Memoria

De la clasificación general al ciclo de vida de los objetos

Resumen: Esta guía presenta las principales formas de clasificar los lenguajes de programación y desarrolla con especial detalle la administración de memoria. Explica la liberación explícita, la recolección por alcanzabilidad, el conteo de referencias y los modelos basados en alcance o propiedad.

Curso: Sistemas Operativos

German Sánchez Torres, Ph.D.

Versión: 1.0

Índice general

1	Taxonomías de los Lenguajes de Programación	1
1.1	Mapa general	1
1.2	Nivel de abstracción	2
1.2.1	Lenguaje máquina y representación binaria	2
1.2.2	Lenguaje ensamblador	2
1.3	Paradigmas de programación	3
1.3.1	Paradigma imperativo y programación procedural	3
1.3.2	Paradigma orientado a objetos	3
1.3.3	Paradigma funcional	4
1.3.4	Paradigma lógico	4
1.3.5	Lenguajes multiparadigma	4
1.4	Forma de ejecución	4
1.5	Sistema de tipos	5
1.6	Propósito	5
1.7	Síntesis del capítulo	6
2	Fundamentos de Administración de Memoria	7
2.1	Vocabulario fundamental	7
2.2	Propiedad, alias y ciclo de vida	8
2.3	Alcance y tiempo de vida	8
2.4	Criterio de clasificación	8
2.5	Propiedades transversales	9
2.6	Pila, memoria dinámica y recursos	10
2.7	Síntesis del capítulo	10
3	Administración Manual de Memoria	11
3.1	Uso correcto en C	11
3.2	Mapa de errores	12
3.3	Fuga de memoria	12
3.4	Doble liberación	13
3.5	Puntero colgante y uso después de liberar	13
3.6	Otros errores relevantes	13
3.7	Ventajas, costos y contexto	14
3.8	Diagnóstico con AddressSanitizer	14
4	Recuperación Automática por Referencias o Alcanzabilidad	15
4.1	Recolección por trazado	15
4.1.1	Ejemplo en Java	16
4.2	Estrategias de recolección	16
4.3	Conteo de referencias	17
4.3.1	Ciclo de referencias en CPython	17
4.3.2	Conteo automático de referencias en Swift	18

4.4	Comparación interna	18
4.5	Síntesis del capítulo	18
5	Liberación Basada en Alcance o Propiedad	21
5.1	RAII en C++ moderno	21
5.1.1	Propiedad compartida	22
5.2	Propiedad y préstamos en Rust	22
5.2.1	Movimiento de propiedad	22
5.3	Memoria automática y recursos explícitos	23
5.4	Comparación entre C++ moderno y Rust	23
5.5	Síntesis del capítulo	24
6	Comparación de Mecanismos y Criterios de Selección	25
6.1	Comparación general	25
6.2	Conceptos que deben mantenerse separados	26
6.2.1	Seguridad de memoria y recuperación automática	26
6.2.2	Liberación determinista y liberación automática	26
6.2.3	Memoria y otros recursos	26
6.2.4	Pila y memoria dinámica	27
6.3	Precisiones conceptuales frecuentes	27
6.4	Criterios de selección	27
6.5	Matriz de decisión resumida	28
7	Prácticas de Consolidación	29
7.1	Diagnóstico de errores manuales	29
7.2	Reescritura mediante RAI	29
7.3	Movimiento y préstamo en Rust	30
7.4	Ciclos y referencias débiles	30
7.5	Análisis transversal de un lenguaje	30
7.6	Preguntas de análisis	31
7.7	Rúbrica sugerida	31
8	Conclusiones	33

Índice de figuras

1.1	Dimensiones principales para clasificar lenguajes de programación.	2
1.2	Principales rutas desde el programa fuente hasta la ejecución.	5
2.1	Ciclo de vida conceptual de un objeto dinámico.	8
2.2	Clasificación por el evento que desencadena la recuperación.	9
3.1	Relación entre errores temporales, espaciales y de propiedad.	12
4.1	Alcanzabilidad desde las raíces y ciclo aislado recuperable.	15

Índice de tablas

1.1	Clasificación según el nivel de abstracción.	2
1.2	Paradigmas principales y lenguajes representativos.	3
2.1	Conceptos fundamentales de memoria.	7
3.1	Balance de la administración manual.	14
4.1	Aportes y costos de la recolección por trazado.	17
4.2	Recolección por trazado y conteo de referencias.	18
5.1	Punteros inteligentes principales de C++.	22
5.2	Correspondencia entre problemas manuales y controles de Rust seguro.	23
5.3	RAII de C++ y propiedad de Rust.	23
6.1	Evento de recuperación y grado de automatización.	25
6.2	Comportamiento temporal y riesgos característicos.	26
6.3	Lenguajes e implementaciones representativos.	26
6.4	Reformulación de afirmaciones imprecisas.	27
6.5	Adecuación cualitativa según necesidades generales.	28
7.1	Criterios para evaluar las prácticas.	31

Índice de códigos

1.1	Suma elemental en ensamblador x86.	3
1.2	Actualización de estado en el paradigma imperativo.	3
1.3	Estado y comportamiento reunidos en un objeto.	3
1.4	Transformación funcional de una lista.	4
1.5	Relación recursiva de ascendencia en Prolog.	4
3.1	Reserva, comprobación, uso y liberación en C.	11
3.2	Pérdida de la dirección de un bloque reservado.	12
3.3	Doble liberación y comportamiento indefinido.	13
3.4	Alias convertido en puntero colgante.	13
3.5	Compilación y ejecución con AddressSanitizer.	14
4.1	Pérdida de la última referencia a una lista en Java.	16
4.2	Ciclo de referencias entre dos nodos.	17
4.3	Ruptura de un posible ciclo fuerte en Swift.	18
5.1	Propiedad exclusiva mediante <code>std::unique_ptr</code>	21
5.2	Préstamo inmutable sin transferencia de propiedad.	22
5.3	Transferencia de propiedad mediante movimiento.	22
5.4	Cierre determinista de un archivo en Python.	23

Capítulo 1

Taxonomías de los Lenguajes de Programación

Una taxonomía organiza elementos mediante criterios definidos. En los lenguajes de programación, los principales criterios consideran la cercanía al hardware, la forma de expresar soluciones, el sistema de tipos, el modelo de ejecución y la administración de memoria [11, 13].

No existe una categoría única capaz de describir por completo un lenguaje. Cada clasificación observa una dimensión diferente. Java, por ejemplo, puede describirse simultáneamente como lenguaje de alto nivel, orientado a objetos, de tipado estático y con recuperación automática de memoria.

Python puede caracterizarse como lenguaje de alto nivel, de propósito general, multiparadigma, de tipado dinámico y con administración automática de memoria. Estas clasificaciones no forman una escala de calidad; cada decisión ofrece ventajas y costos según el problema y la implementación.

Un lenguaje ocupa simultáneamente una posición en varias dimensiones. El paradigma, el sistema de tipos, la forma de ejecución y la administración de memoria no son clasificaciones excluyentes.

1.1 Mapa general

La Figura 1.1 resume las dimensiones principales. Las ramas describen perspectivas complementarias sobre un mismo lenguaje.

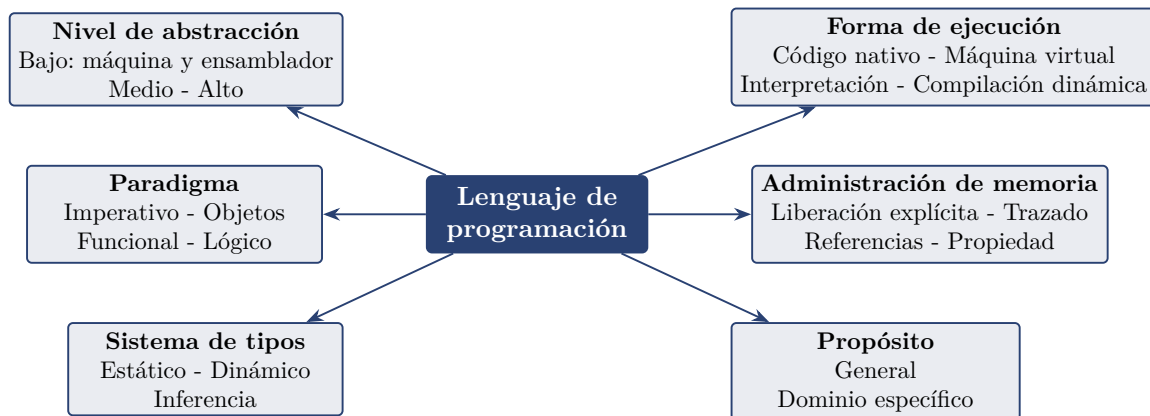


Figura 1.1: Dimensiones principales para clasificar lenguajes de programación.

1.2 Nivel de abstracción

El nivel de abstracción indica cuántos detalles de la máquina debe manejar directamente el programador. Las abstracciones permiten representar conceptos del dominio -como colecciones, imágenes o entidades financieras- sin operar directamente sobre registros, direcciones e instrucciones del procesador.

Tabla 1.1: Clasificación según el nivel de abstracción.

Nivel	Subcategoría	Descripción	Ejemplos
Bajo	Lenguaje máquina	Secuencias binarias ejecutadas directamente por el procesador.	Código máquina para arquitecturas x86-64, Arm o RISC-V.
Bajo	Ensamblador	Representación simbólica cercana a las instrucciones de una arquitectura.	Ensamblador para x86-64, Arm o RISC-V.
Medio	–	Abstracciones estructuradas con acceso directo al hardware.	C y C++.
Alto	–	Oculta gran parte de los detalles de la máquina.	Python, Java, C# y JavaScript.

1.2.1 Lenguaje máquina y representación binaria

x86-64, Arm y RISC-V designan arquitecturas de conjunto de instrucciones: cada una define las operaciones, los registros y las codificaciones binarias disponibles. Un procesador ejecuta patrones de bits. La secuencia 10110000 01100001, por ejemplo, puede representar una instrucción y su operando en una arquitectura determinada. Los bits no poseen un significado universal: su interpretación depende del conjunto de instrucciones del procesador [9].

El lenguaje máquina se ubica en el nivel más bajo de la clasificación. Es ejecutable directamente, pero el desarrollo de programas extensos mediante secuencias binarias resulta impráctico y propenso a errores.

1.2.2 Lenguaje ensamblador

El ensamblador reemplaza patrones binarios por nombres simbólicos. Una herramienta denominada también *ensamblador* traduce estas instrucciones a código máquina.

```

1 mov eax, 2
2 add eax, 3

```

Código 1.1: Suma elemental en ensamblador x86.

El fragmento modifica un registro del procesador y depende de una arquitectura específica. La expresión “nivel medio” se utiliza de manera informal para C y C++, pues ofrecen funciones y estructuras, pero conservan acceso a direcciones y memoria. Un mayor nivel de abstracción no implica necesariamente menor rendimiento: también influyen el algoritmo, el compilador y el hardware.

1.3 Paradigmas de programación

Un paradigma establece una forma de concebir y expresar la solución de un problema. No se limita a la sintaxis: determina qué elementos se consideran centrales y cómo se relacionan [11, 13].

Tabla 1.2: Paradigmas principales y lenguajes representativos.

Paradigma	Idea principal	Ejemplos
Imperativo	Describe cómo modificar el estado mediante una secuencia de instrucciones.	C, Pascal, Go.
Orientado a objetos	Organiza el software mediante objetos con estado y comportamiento.	Java, C#, Smalltalk.
Funcional	Prioriza funciones, composición e inmutabilidad.	Haskell, Lisp, Elixir.
Lógico	Declara hechos y reglas para que el sistema derive respuestas.	Prolog.

1.3.1 Paradigma imperativo y programación procedural

El paradigma imperativo entiende el programa como una secuencia de órdenes que modifica un estado. La programación procedural es una forma de este paradigma que organiza instrucciones en procedimientos o funciones reutilizables.

```

1 int suma = 0;
2 for (int i = 1; i <= 3; i++)
3     suma += i;
4 printf("%d\n", suma);

```

Código 1.2: Actualización de estado en el paradigma imperativo.

1.3.2 Paradigma orientado a objetos

El sistema se modela mediante objetos que reúnen estado y comportamiento. La encapsulación protege las reglas internas; la composición, la herencia y el polimorfismo permiten establecer relaciones y sustituir comportamientos.

```

1 class Cuenta:
2     def __init__(self, saldo): self.saldo = saldo
3     def depositar(self, valor): self.saldo += valor

```



```
4 cuenta = Cuenta(100); cuenta.depositar(50)
```

Código 1.3: Estado y comportamiento reunidos en un objeto.

1.3.3 Paradigma funcional

La computación se expresa principalmente mediante evaluación y composición de funciones. Este paradigma favorece datos inmutables y funciones puras, que producen la misma salida para una misma entrada y no modifican estado externo observable.

```
1 cuadrado x = x * x
2 cuadrados = map cuadrado [1, 2, 3]
3 -- Resultado: [1, 4, 9]
```

Código 1.4: Transformación funcional de una lista.

1.3.4 Paradigma lógico

El programador declara hechos y reglas; el motor de inferencia busca valores que satisfagan una consulta.

```
1 progenitor(ana, luis).
2 progenitor(luis, eva).
3 ancestro(X, Z) :- progenitor(X, Z).
4 ancestro(X, Z) :- progenitor(X, Y), ancestro(Y, Z).
5 % Consulta: ancestro(ana, eva).
```

Código 1.5: Relación recursiva de ascendencia en Prolog.

La primera regla cubre una relación directa y la segunda la extiende recursivamente. El motor puede deducir que Ana es ancestro de Eva.

1.3.5 Lenguajes multiparadigma

“Multiparadigma” no designa un paradigma independiente, sino una característica transversal. Python, JavaScript, C++ y Rust permiten combinar varios estilos. Esta flexibilidad no elimina la necesidad de mantener un diseño coherente.

1.4 Forma de ejecución

El código fuente debe convertirse en operaciones realizables por la máquina. Esta dimensión describe cuándo ocurre la traducción y qué intermediarios participan [11].

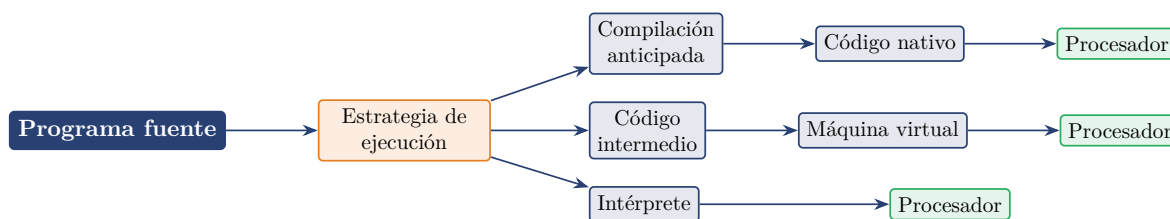


Figura 1.2: Principales rutas desde el programa fuente hasta la ejecución.

Un compilador traduce el programa a otra representación antes de ejecutarlo. El *bytecode* es una forma de código intermedio destinada a ser ejecutada por una máquina virtual. Un intérprete participa durante la ejecución. La compilación JIT (*just in time*, “justo a tiempo”) traduce durante la ejecución las partes seleccionadas para optimización.

La división no es rígida. Java suele compilarse a *bytecode*, que la máquina virtual de Java (JVM, *Java Virtual Machine*) interpreta o compila con JIT. Una implementación habitual de Python compila a *bytecode* y lo ejecuta en una máquina virtual. JavaScript moderno combina interpretación y JIT. Por ello, “compilado” e “interpretado” describen implementaciones con mayor precisión que lenguajes completos.

1.5 Sistema de tipos

Un tipo define qué valores puede representar una expresión y qué operaciones son válidas. Un sistema de tipos intenta detectar operaciones incoherentes y documentar restricciones del programa [10].

Tipado estático. Las comprobaciones ocurren principalmente antes de ejecutar. Ejemplos: C, Java, Rust y Haskell.

Tipado dinámico. Las comprobaciones ocurren principalmente durante la ejecución. Ejemplos: Python, Ruby y JavaScript.

Inferencia de tipos. El compilador deduce tipos que no se escribieron explícitamente. Ejemplos: Rust, Kotlin, Swift y Haskell.

Tipado fuerte o débil.

Son términos informales y ambiguos. Resulta preferible especificar si el lenguaje admite conversiones implícitas concretas.

El tipado estático no obliga a escribir todos los tipos si existe inferencia. El tipado dinámico tampoco implica ausencia de tipos: los valores conservan un tipo, pero muchas comprobaciones se realizan al ejecutar.

1.6 Propósito

Los lenguajes de propósito general, como C, Java, Python y Rust, pueden aplicarse en numerosos dominios. Los lenguajes de dominio específico (DSL, *domain-specific languages*) restringen su campo para expresar con precisión conceptos especializados. SQL (*Structured Query Language*) se orienta a consultas; Verilog, a la descripción de hardware; y GLSL (*OpenGL Shading Language*), a programas que se ejecutan en una unidad de procesamiento gráfico (GPU, *graphics processing unit*).

Un lenguaje especializado no es inferior a uno de propósito general. Su vocabulario y sus restricciones están adaptados a un dominio concreto.

1.7 Síntesis del capítulo

- Las taxonomías describen dimensiones complementarias, no niveles de calidad.
- Lenguaje máquina y ensamblador son formas de bajo nivel.
- Lo procedural es una forma del paradigma imperativo; “multiparadigma” es una característica transversal.
- La forma de ejecución depende de la implementación y puede combinar compilación, interpretación y JIT.
- La administración de memoria constituye una dimensión propia y será el eje de los capítulos siguientes.

Capítulo 2

Fundamentos de Administración de Memoria

La ejecución de un programa requiere memoria para almacenar instrucciones, variables y objetos. Parte de esa memoria posee una duración determinada por la estructura del programa. Otra parte se solicita dinámicamente porque su tamaño o duración solo se conocen durante la ejecución.

La pregunta técnica central no consiste únicamente en identificar quién libera la memoria. También debe establecerse qué evento indica que el objeto ha terminado su vida útil y qué componente hace efectiva la recuperación [5, 11].

2.1 Vocabulario fundamental

Tabla 2.1: Conceptos fundamentales de memoria.

Concepto	Definición
Valor	Información manipulada por el programa, como el número 42 o el texto <code>."da"</code> .
Objeto	Entidad almacenada con tipo y tiempo de vida. Según el lenguaje, su identidad puede ser observable o no.
Referencia o puntero	Medio para localizar o acceder a un objeto. Un puntero suele exponer una dirección; una referencia puede ocultarla y aplicar restricciones.
Reserva de memoria	Obtención de espacio para almacenar un objeto. No equivale a asignar un valor mediante una operación como <code>x = 5</code> .
Tiempo de vida	Intervalo durante el cual el objeto existe y puede utilizarse válidamente.
Liberación	Finalización del objeto y devolución o reutilización del espacio que ocupaba.
Propiedad	Responsabilidad de mantener vivo y liberar un recurso.
Alias	Nombre, referencia o puntero adicional que permite acceder al mismo objeto.

La dificultad principal aparece cuando varias referencias acceden al mismo objeto. La vida de las referencias y la vida del objeto no necesariamente terminan al mismo tiempo [3].

2.2 Propiedad, alias y ciclo de vida

La Figura 2.1 resume las etapas de un objeto dinámico. El propietario controla su vida; los alias permiten acceso, pero no deberían asumir automáticamente la responsabilidad de liberación.

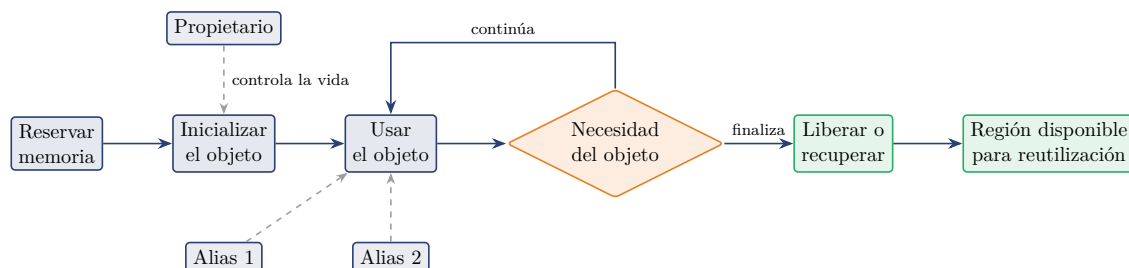


Figura 2.1: Ciclo de vida conceptual de un objeto dinámico.

Considérese un puntero **p** y otro puntero **alias** dirigidos al mismo objeto. Si **p** libera el objeto en un sistema manual, **alias** no desaparece. Permanece dirigido a una región cuya vida terminó. Esta separación origina punteros colgantes y usos posteriores a la liberación.

2.3 Alcance y tiempo de vida

El **alcance** es la región del código donde un nombre puede utilizarse. El **tiempo de vida** es el periodo de ejecución durante el cual el objeto existe. Los conceptos están relacionados, pero no son equivalentes [11].

Un nombre puede salir de alcance mientras el objeto continúa vivo porque otra referencia lo conserva. En lenguajes que permiten operaciones inseguras, un puntero puede permanecer dentro del alcance después de que el objeto señalado haya sido destruido.

El alcance es una propiedad del nombre en el programa; el tiempo de vida es una propiedad del objeto durante la ejecución.

2.4 Criterio de clasificación

La clasificación utilizada en esta guía se basa en el **evento que autoriza o desencadena la recuperación** [5, 7, 15]:

1. una orden explícita del programador, como **free** o **delete**;
2. la salida del alcance o el final de la propiedad de un objeto;

3. la reducción del contador de referencias fuertes a cero;
4. la pérdida de alcanzabilidad desde las raíces del programa.

En C++, RAII (*Resource Acquisition Is Initialization*, “la adquisición de recursos es inicialización”) vincula el recurso con la vida de un objeto: su construcción establece la propiedad y su destructor libera el recurso al finalizar esa vida. El Capítulo 5 desarrolla este mecanismo y lo compara con el modelo de propiedad de Rust.

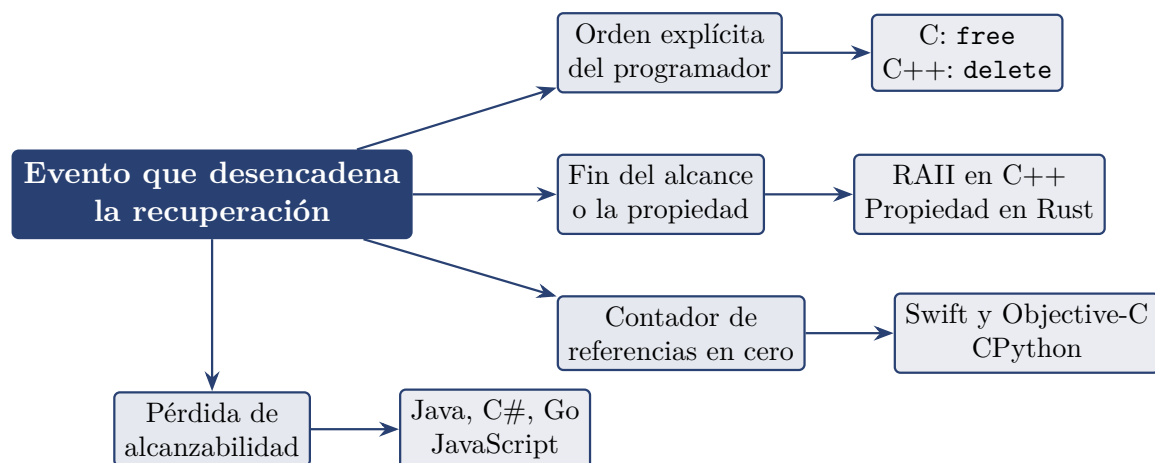


Figura 2.2: Clasificación por el evento que desencadena la recuperación.

2.5 Propiedades transversales

Los mecanismos anteriores pueden describirse mediante propiedades que se superponen:

Manual o automático.

Indica si el programa debe ordenar explícitamente cada liberación o si el compilador o el entorno la realiza a partir de una regla.

Determinista o no determinista.

Indica si puede predecirse el punto en el que ocurre la liberación.

Simple o combinado.

Indica si una implementación utiliza un mecanismo principal o integra varios.

El conteo de referencias es automático y normalmente determinista. Un recolector por trazado es automático, pero no suele garantizar el instante exacto. RAII automatiza una liberación determinista al finalizar el alcance. La liberación explícita permite seleccionar el punto de ejecución, pero solo ocurre si el programa alcanza y ejecuta la operación correcta [5, 15].

Las implementaciones pueden combinar mecanismos. CPython, la implementación de referencia de Python escrita principalmente en C, integra conteo de referencias y recolección de ciclos [2]; C++ permite liberación explícita y RAII; los lenguajes con recolección automática de memoria (GC, por *garbage collection*) suelen exigir gestión explícita de archivos y conexiones.

2.6 Pila, memoria dinámica y recursos

La pila suele almacenar marcos de llamadas y variables locales con vida ligada al alcance. La memoria dinámica permite que los datos vivan independientemente de una llamada. Esta distinción es un modelo útil, no una regla universal: el compilador puede conservar un valor en un registro, eliminar una reserva o trasladar un objeto a otra región [11].

Reservar, inicializar y liberar son operaciones diferentes. En C, `malloc` reserva bytes pero no construye un valor inicial significativo. En C++, `new` suele combinar reserva y construcción; `delete` combina destrucción y liberación [4, 12].

La memoria tampoco es el único recurso. Archivos, sockets, bloqueos y conexiones necesitan cierre oportuno. Un recolector puede recuperar la memoria del objeto que los representa sin garantizar la liberación temporalmente adecuada del recurso externo.

2.7 Síntesis del capítulo

- Propiedad, alias, alcance y tiempo de vida explican la mayoría de los problemas de administración de memoria.
- La clasificación principal se basa en el evento de recuperación, no en categorías superpuestas como “automático” y “determinista”.
- Los lenguajes y sus implementaciones pueden combinar mecanismos.
- Seguridad de memoria, recuperación automática y gestión de recursos son problemas relacionados, pero distintos.

Capítulo 3

Administración Manual de Memoria

En la administración manual, el programa solicita memoria y debe liberarla explícitamente. C utiliza `malloc/free`; C++ conserva `new/delete`, aunque el C++ moderno favorece RAII y punteros inteligentes [4, 12, 15].

El desafío central no se limita a recordar una llamada a `free`. Se necesita mantener una regla correcta de propiedad a través de funciones, estructuras de datos, rutas de error y ejecución concurrente. Para cada bloque deben definirse:

1. el componente propietario;
2. las partes del programa que conservan alias;
3. el punto de ejecución en el que la liberación es segura.

3.1 Uso correcto en C

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void) {
5      size_t cantidad = 5;
6      int *numeros = malloc(cantidad * sizeof *numeros);
7
8      if (numeros == NULL) {
9          return EXIT_FAILURE;
10     }
11
12     for (size_t i = 0; i < cantidad; i++) {
13         numeros[i] = (int)(i * i);
14         printf("%d\n", numeros[i]);
15     }
16
17     free(numeros);
18     numeros = NULL;
19     return EXIT_SUCCESS;
20 }
```

Código 3.1: Reserva, comprobación, uso y liberación en C.

El programa comprueba el resultado de `malloc`, utiliza únicamente los elementos reservados y libera el bloque una vez. `NULL` representa aquí un puntero nulo, es decir, un valor que no señala un objeto válido. Asignarlo al puntero reduce algunos usos accidentales posteriores, pero no modifica otros alias existentes.

3.2 Mapa de errores

Los errores de memoria manual pueden organizarse en tres familias. Los errores temporales utilizan un objeto fuera de su tiempo de vida; los espaciales exceden los límites de una región; los errores de propiedad afectan la responsabilidad de liberación [12, 14].

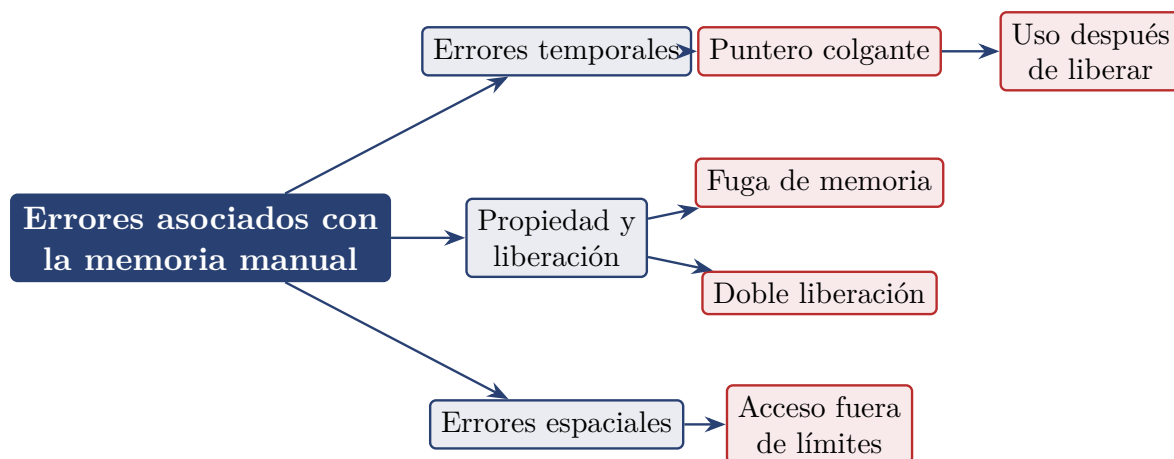


Figura 3.1: Relación entre errores temporales, espaciales y de propiedad.

3.3 Fuga de memoria

Una fuga ocurre cuando un bloque reservado deja de poder liberarse o permanece retenido sin necesidad. El espacio continúa ocupado hasta que finaliza el proceso.

Se pierde la última referencia utilizable al bloque antes de ejecutar la liberación correspondiente.

```

1 void fuga(void) {
2     int *datos = malloc(100 * sizeof *datos);
3     if (datos == NULL) return;
4
5     datos = NULL; // La dirección original se ha perdido.
6 }
  
```

Código 3.2: Pérdida de la dirección de un bloque reservado.

El impacto depende del ciclo de vida del proceso. En una utilidad breve puede pasar inadvertido; en un servidor, servicio o sistema embebido puede producir crecimiento sostenido del consumo.

Una fuga no exige perder la dirección. En sistemas con recolección automática puede existir **retención innecesaria**: una caché o colección conserva referencias a objetos que ya no tienen utilidad. El recolector observa alcanzabilidad, no intención.

3.4 Doble liberación

La doble liberación ocurre cuando dos operaciones intentan devolver el mismo bloque.

```
1 int *p = malloc(sizeof *p);
2 if (p != NULL) {
3     free(p);
4     free(p);      // Error: comportamiento indefinido.
5 }
```

Código 3.3: Doble liberación y comportamiento indefinido.

El error puede corromper el gestor de memoria, provocar una terminación inesperada o crear una vulnerabilidad. Asignar NULL después de `free` ayuda en casos básicos, pero no resuelve el problema cuando existen otros alias.

Una doble liberación suele indicar propiedad ambigua: dos componentes consideran que son responsables del mismo bloque.

3.5 Puntero colgante y uso después de liberar

Un puntero queda colgante cuando señala una región cuyo objeto ya terminó su vida. El puntero puede existir sin producir inmediatamente un error; el fallo ocurre cuando se accede a la región mediante él, lo que constituye un *use-after-free*.

```
1 int *p = malloc(sizeof *p);
2 if (p != NULL) {
3     *p = 42;
4     int *alias = p;
5     free(p);
6     printf("%d\n", *alias); // Uso después de liberar.
7 }
```

Código 3.4: Alias convertido en puntero colgante.

La memoria podría conservar temporalmente el valor anterior, ser reutilizada por otro objeto o quedar protegida por el sistema. Por ello, el comportamiento es indefinido y puede variar entre ejecuciones.

3.6 Otros errores relevantes

Acceso fuera de límites.

Lectura o escritura antes del inicio o después del final de una región válida.

Memoria no inicializada.

Uso de bytes cuyo contenido no representa todavía un valor válido para el programa.

Familias de reserva incompatibles.

Combinación incorrecta de operaciones, como `new` con `free` o `malloc` con `delete`.

Desbordamiento aritmético.

Cálculo incorrecto del tamaño solicitado, por ejemplo al multiplicar cantidad y tamaño sin comprobar el límite.

3.7 Ventajas, costos y contexto

La memoria manual ofrece control explícito sobre la reserva y la liberación. Permite construir un comportamiento temporal predecible cuando el asignador y el diseño completo también son predecibles. Este control resulta útil en sistemas operativos, controladores, sistemas embebidos y determinadas aplicaciones de tiempo real.

No garantiza por sí misma mayor velocidad ni latencia constante. El asignador, la fragmentación, la localidad de los datos, la concurrencia y los errores de diseño también afectan el resultado [16].

Una ABI (*application binary interface*, interfaz binaria de aplicaciones) define las convenciones que permiten interoperar a funciones, bibliotecas y herramientas compiladas.

Tabla 3.1: Balance de la administración manual.

Aportes	Costos
Control explícito del ciclo de vida.	Mayor carga de razonamiento sobre propiedad y alias.
Posibilidad de evitar un recolector global.	Fugas, doble liberación y usos posteriores a liberar.
Adecuación a entornos con recursos limitados.	Errores difíciles de reproducir y riesgos de seguridad.
Integración directa con hardware y ABI.	Rutas de error y concurrencia más complejas.

3.8 Diagnóstico con AddressSanitizer

AddressSanitizer instrumenta el programa para detectar accesos fuera de límites, usos después de liberar y dobles liberaciones [14].

```
cc -fsanitize=address -g ejemplo.c -o ejemplo
./ejemplo
```

Código 3.5: Compilación y ejecución con AddressSanitizer.

Compile por separado los ejemplos erróneos de este capítulo. Compare el diagnóstico, la línea informada y la secuencia de llamadas asociada a cada error. No combine todos los fallos en una sola ejecución, pues el primero puede finalizar el programa.

Capítulo 4

Recuperación Automática por Referencias o Alcanzabilidad

En la recuperación automática, el entorno de ejecución o el código insertado por el compilador aplica una política para decidir cuándo ciertos objetos pueden destruirse. El programador deja de indicar manualmente cada liberación, pero conserva la responsabilidad de diseñar estructuras con un ciclo de vida razonable.

Automático no significa memoria infinita, recuperación instantánea ni cierre automático de todo recurso. Los dos mecanismos principales estudiados en este capítulo son la recolección por trazado y el conteo de referencias [1, 5].

4.1 Recolección por trazado

Un recolector por trazado parte de un conjunto de **raíces**: variables locales activas, variables globales, referencias internas de la máquina virtual y otros puntos definidos por la implementación. Sigue las referencias y marca los objetos alcanzables. Los objetos fuera de ese grafo pueden recuperarse [5].

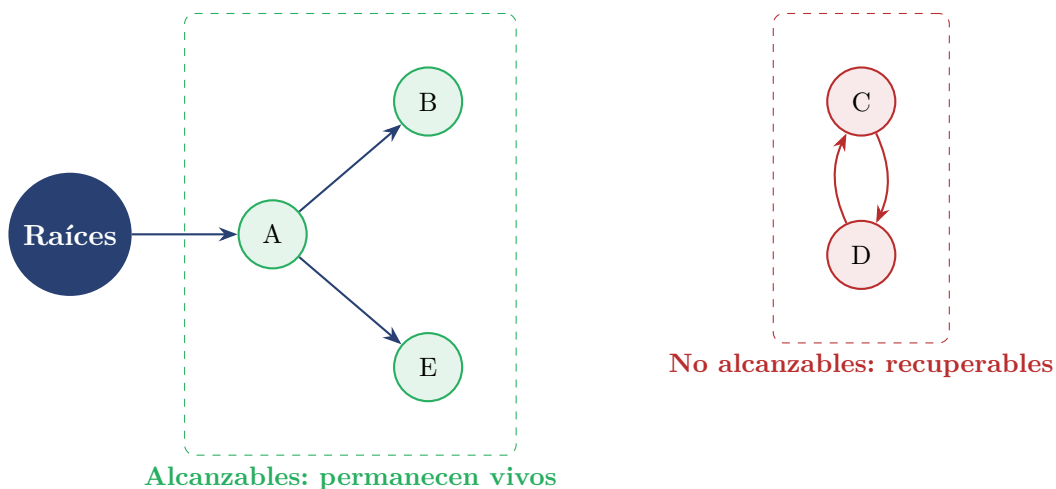


Figura 4.1: Alcanzabilidad desde las raíces y ciclo aislado recuperable.

Aunque C y D formen un ciclo, no existe un camino desde las raíces hasta ellos. Un recolector por trazado puede identificarlos como recuperables.

El recolector conoce la alcanzabilidad, no la utilidad semántica. Una caché global puede mantener alcanzable un objeto que el programa no volverá a consultar.

4.1.1 Ejemplo en Java

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class MemoriaAutomatica {
5     public static void main(String[] args) {
6         List<String> nombres = new ArrayList<>();
7         nombres.add("Ada");
8         nombres.add("Grace");
9
10        nombres = null; // La lista deja de ser alcanzable.
11        // No se exige el instante exacto de recoleccion.
12    }
13 }
```

Código 4.1: Pérdida de la última referencia a una lista en Java.

La lista puede recuperarse cuando deja de ser alcanzable. Los literales de texto poseen un tratamiento propio de la plataforma y no deben confundirse con el objeto `ArrayList` del ejemplo.

4.2 Estrategias de recolección

Los recolectores modernos suelen combinar varias estrategias [5]:

Marcado y barrido. Marca los objetos alcanzables y recupera los restantes.

Copia. Traslada los objetos vivos a otra región y descarta en conjunto la región anterior.

Recolección generacional.

Separa los objetos por edad y revisa con mayor frecuencia las generaciones jóvenes.

Compactación.

Reubica objetos vivos para reducir fragmentación y formar espacio contiguo.

Trabajo incremental o concurrente.

Distribuye parte del trabajo o lo ejecuta junto con la aplicación para reducir pausas extensas.

Tabla 4.1: Aportes y costos de la recolección por trazado.

Aportes	Costos o límites
Recupera ciclos no alcanzables.	Trabajo adicional del entorno de ejecución.
Evita dobles liberaciones ordinarias.	El instante de recuperación suele ser no determinista.
Simplifica estructuras con propiedad compartida.	Puede introducir pausas o consumo adicional de memoria.
Reduce ciertos errores temporales.	No impide la retención de objetos todavía alcanzables.

Un GC no garantiza por sí solo seguridad de memoria completa. El lenguaje debe impedir además accesos fuera de límites, operaciones nativas inseguras y otras formas de corrupción [14].

4.3 Conteo de referencias

Cada objeto conserva un contador de referencias fuertes. Crear o copiar una referencia incrementa el contador; eliminarla lo reduce. Cuando el contador llega a cero, el objeto suele destruirse inmediatamente.

La liberación es normalmente determinista y no requiere recorrer todo el grafo para cada objeto. A cambio, las actualizaciones del contador consumen tiempo y los ciclos de referencias necesitan tratamiento especial [1, 5].

4.3.1 Ciclo de referencias en CPython

CPython utiliza conteo de referencias como mecanismo principal y lo complementa con un recolector de ciclos [2].

```

1  class Nodo:
2      def __init__(self, nombre):
3          self.nombre = nombre
4          self.siguiente = None
5
6  a = Nodo("A")
7  b = Nodo("B")
8  a.siguiente = b
9  b.siguiente = a
10
11 del a
12 del b

```

Código 4.2: Ciclo de referencias entre dos nodos.

Un conteo puro no puede recuperar el ciclo porque ambos contadores permanecen por encima de cero. El recolector complementario de CPython detecta que el ciclo completo es inaccesible desde el resto del programa.

4.3.2 Conteo automático de referencias en Swift

Swift emplea ARC (*Automatic Reference Counting*, “conteo automático de referencias”). El compilador inserta las operaciones necesarias para mantener contadores; las referencias **weak** y **unowned** permiten expresar relaciones que no deben mantener vivo al objeto [8].

```

1  final class Persona {
2      let nombre: String
3      var vivienda: Vivienda?
4
5      init(nombre: String) {
6          self.nombre = nombre
7      }
8  }
9
10 final class Vivienda {
11     weak var residente: Persona?
12 }
13
14 let ana = Persona(nombre: "Ana")
15 let casa = Vivienda()
16 ana.vivienda = casa           // Referencia fuerte
17 casa.residente = ana         // Referencia debil

```

Código 4.3: Ruptura de un posible ciclo fuerte en Swift.

Persona mantiene la vivienda mediante una referencia fuerte; **Vivienda** se refiere a su residente mediante una referencia débil. Si ambas relaciones fueran fuertes, podrían mantenerse mutuamente después de desaparecer las referencias externas.

El conteo de referencias evita las pausas de trazado características de ciertos recolectores, pero no elimina toda latencia: liberar un objeto puede desencadenar una cascada de destrucciones [5].

4.4 Comparación interna

Tabla 4.2: Recolección por trazado y conteo de referencias.

Aspecto	Trazado	Conteo de referencias
Evento	Pérdida de alcanzabilidad.	Contador fuerte igual a cero.
Ciclos aislados	Se recuperan.	Requieren mecanismo adicional.
Momento de liberación	Normalmente no garantizado.	Normalmente inmediato.
Costo característico	Recorrido periódico del grafo.	Actualización frecuente de contadores.
Retención innecesaria	Posible si el objeto sigue alcanzable.	Posible por ciclos o referencias conservadas.

4.5 Síntesis del capítulo

- El trazado recupera objetos que ya no son alcanzables, incluso si forman ciclos.
- El conteo de referencias suele liberar inmediatamente al llegar a cero, pero no resuelve ciclos por sí solo.
- CPython combina conteo y recolección de ciclos; Swift utiliza ARC y referencias no fuertes.

- La recuperación automática reduce trabajo manual, pero no conoce la utilidad semántica ni gestiona necesariamente recursos externos.

Capítulo 5

Liberación Basada en Alcance o Propiedad

La liberación basada en alcance vincula el ciclo de vida del recurso con una estructura que el compilador puede observar. En C++ moderno, RAII asocia el recurso con la vida de un objeto. Rust incorpora propiedad, movimientos y préstamos al sistema de tipos. Ambos enfoques permiten recuperación automática y determinista sin exigir un recolector por trazado [7, 15].

5.1 RAII en C++ moderno

RAII (*Resource Acquisition Is Initialization*, “la adquisición de recursos es inicialización”) adquiere el recurso durante la construcción de un objeto y lo libera durante su destrucción. En otras palabras, convierte el tiempo de vida del objeto en la regla que controla el recurso. Al salir del alcance, el lenguaje ejecuta automáticamente los destructores, incluso ante retornos tempranos o excepciones [4, 15].

```
1 #include <iostream>
2 #include <memory>
3
4 int main() {
5     auto numero = std::make_unique<int>(42);
6     std::cout << *numero << '\n';
7 } // El unique_ptr sale de alcance y libera la memoria.
```

Código 5.1: Propiedad exclusiva mediante `std::unique_ptr`.

`std::unique_ptr` expresa propiedad exclusiva. No puede copiarse, pero sí moverse, de modo que la responsabilidad pasa de un propietario a otro sin duplicarse.

RAII no es un recolector de basura. La liberación se deriva del alcance y de los destructores, no de un recorrido que busque objetos inalcanzables.

5.1.1 Propiedad compartida

`std::shared_ptr` emplea conteo de referencias para casos donde varios componentes comparten propiedad. Debe utilizarse solo cuando el modelo realmente necesita esa relación. Dos `shared_ptr` que se referencian mutuamente forman un ciclo; `std::weak_ptr` permite observar un objeto sin incrementar el contador fuerte [4, 15].

Tabla 5.1: Punteros inteligentes principales de C++.

Tipo	Propiedad	Uso principal
<code>unique_ptr</code>	Exclusiva.	Opción predeterminada para un recurso con un único dueño.
<code>shared_ptr</code>	Compartida mediante contador.	Objetos cuya vida depende genuinamente de varios dueños.
<code>weak_ptr</code>	Observación no propietaria.	Ruptura de ciclos y acceso condicional a un objeto compartido.

5.2 Propiedad y préstamos en Rust

Rust comprueba en compilación reglas de propiedad, movimiento, préstamos y tiempos de vida [6, 7]. Cada valor posee un propietario; al salir este de alcance, se ejecuta la operación de destrucción correspondiente. Una transferencia puede mover el valor, mientras que un préstamo ofrece acceso temporal sin transferir la propiedad.

```

1 fn main() {
2     let mensaje = String::from("hola");
3     imprimir(&mensaje);      // Prestamo: no transfiere propiedad.
4     println!("{mensaje}");   // El valor sigue siendo válido.
5 } // El propietario sale de alcance.
6
7 fn imprimir(texto: &str) {
8     println!("{texto}");
9 }

```

Código 5.2: Préstamo inmutable sin transferencia de propiedad.

El compilador impide en código seguro numerosos usos después de liberar y evita carreras de datos al restringir préstamos mutables concurrentes. Estas garantías trasladan parte del trabajo desde la ejecución hacia la compilación [6].

5.2.1 Movimiento de propiedad

```

1 fn main() {
2     let original = String::from("dato");
3     let nuevo_dueno = original;
4
5     println!("{nuevo_dueno}");
6     // println!("{original}"); // No compila: fue movido.
7 }

```

Código 5.3: Transferencia de propiedad mediante movimiento.

La variable `original` deja de ser válida tras el movimiento. Esta regla evita que dos variables intenten liberar el mismo recurso como propietarias independientes.

Tabla 5.2: Correspondencia entre problemas manuales y controles de Rust seguro.

Problema	Control aplicado
Doble liberación	Un valor posee un propietario activo; los movimientos invalidan el nombre anterior.
Uso después de liberar	Los préstamos no pueden superar el tiempo de vida del objeto.
Alias mutable conflictivo	No pueden coexistir un préstamo mutable y otros préstamos activos incompatibles.
Carrera de datos	Las reglas de alias y los tipos utilizados entre hilos se comprueban en compilación.

Rust conserva un bloque `unsafe` para operaciones que el compilador no puede verificar, como interoperabilidad con C o acceso de bajo nivel. La responsabilidad de mantener las invariantes se desplaza entonces al programador.

5.3 Memoria automática y recursos explícitos

Los lenguajes con GC administran memoria, pero archivos, sockets, bloqueos y conexiones requieren cierre oportuno. Por ello ofrecen construcciones ligadas al alcance.

```

1 with open("datos.txt", encoding="utf-8") as archivo:
2     contenido = archivo.read()
3 # El archivo se cierra incluso si ocurre una excepción.
```

Código 5.4: Cierre determinista de un archivo en Python.

Java ofrece `try-with-resources`; C#, `using`; C++, RAI; y Python, administradores de contexto. Estas construcciones resuelven un problema distinto de la recuperación de memoria: garantizan el final oportuno de un recurso externo [2, 15].

5.4 Comparación entre C++ moderno y Rust

Tabla 5.3: RAI de C++ y propiedad de Rust.

Aspecto	C++ moderno	Rust seguro
Unidad central	Objeto RAI y destructor.	Propietario, movimiento y préstamo.
Liberación	Al finalizar el alcance del objeto propietario.	Al finalizar el alcance del propietario.
Alias	Punteros, referencias y punteros inteligentes.	Préstamos comprobados por el compilador.
Propiedad compartida	<code>shared_ptr</code> y <code>weak_ptr</code> .	<code>Rc/Arc</code> y referencias débiles.
Escape de seguridad	Punteros crudos y operaciones no disciplinadas.	Bloques <code>unsafe</code> .
Compatibilidad histórica	Permite estilos manuales y modernos.	Las reglas seguras son parte central del lenguaje.

5.5 Síntesis del capítulo

- RAII vincula adquisición y liberación con la construcción y destrucción de objetos.
- `unique_ptr` expresa propiedad exclusiva; `shared_ptr`, propiedad compartida; `weak_ptr`, observación no propietaria.
- Rust convierte propiedad y préstamos en reglas verificadas por el compilador.
- Los recursos externos requieren liberación determinista incluso en lenguajes con GC.

Capítulo 6

Comparación de Mecanismos y Criterios de Selección

Ningún mecanismo es universalmente superior. La selección depende del control temporal requerido, las garantías del lenguaje, el presupuesto de memoria, la latencia admisible, la interoperabilidad y la experiencia del equipo [5, 11].

6.1 Comparación general

Tabla 6.1: Evento de recuperación y grado de automatización.

Mecanismo	Evento de recuperación	Automatización
Liberación explícita	El programa ejecuta la operación.	Manual.
Trazado	El objeto deja de ser alcanzable desde las raíces.	Automática.
Conteo de referencias	El contador de referencias fuertes llega a cero.	Automática.
RAII	El objeto propietario sale de alcance.	Automática después de adquirir el recurso.
Propiedad y préstamos	Finaliza el alcance del propietario.	Automática y verificada por el compilador.
Combinación	Intervienen varios eventos.	Depende de los mecanismos integrados.

Tabla 6.2: Comportamiento temporal y riesgos característicos.

Mecanismo	Momento de recuperación	Riesgo característico
Liberación explícita	En el punto indicado, si la operación se ejecuta.	Fugas, dobles liberaciones y punteros colgantes.
Trazado	El instante exacto suele no estar garantizado.	Pausas y retención de objetos alcanzables.
Conteo de referencias	Normalmente inmediato al llegar a cero.	Ciclos y costo de actualizar contadores.
RAII	Determinista al finalizar el alcance.	Uso incorrecto de punteros crudos o propiedad compartida.
Propiedad y préstamos	Determinista al finalizar la propiedad.	Complejidad conceptual y restricciones de préstamo.
Combinación	Depende de cada componente.	Interacción entre modelos de vida.

Tabla 6.3: Lenguajes e implementaciones representativos.

Mecanismo	Ejemplos representativos
Liberación explícita	C y C++ tradicional.
Trazado	Java, C#, Go y JavaScript.
Conteo de referencias	Swift y Objective-C con ARC; CPython como mecanismo principal.
RAII	C++ moderno.
Propiedad y préstamos	Rust.
Combinación	CPython, C++, interoperabilidad con bibliotecas nativas.

6.2 Conceptos que deben mantenerse separados

6.2.1 Seguridad de memoria y recuperación automática

La seguridad de memoria busca impedir accesos fuera de límites, usos posteriores a la liberación y otras operaciones inválidas. Un recolector evita ciertos errores temporales, pero la seguridad completa depende además de las reglas del lenguaje y de sus interfaces con código nativo. Rust puede ofrecer seguridad de memoria en su subconjunto seguro sin utilizar un GC por trazado [6, 14].

6.2.2 Liberación determinista y liberación automática

Una liberación determinista ocurre en un punto predecible, como la salida de un alcance con RAI. Un GC por trazado es automático, pero normalmente no promete el instante exacto. El conteo de referencias suele ser automático y determinista cuando el contador llega a cero.

6.2.3 Memoria y otros recursos

Un recolector puede recuperar memoria sin garantizar el cierre oportuno de archivos, conexiones o bloqueos. La gestión determinista de recursos sigue siendo necesaria.

6.2.4 Pila y memoria dinámica

La pila suele asociarse con llamadas y variables locales; la memoria dinámica, con objetos cuya duración se decide durante la ejecución. Estas ubicaciones son decisiones de implementación. El compilador puede mantener valores en registros, eliminar reservas o trasladar objetos cuando conserva el comportamiento observable.

6.3 Precisiones conceptuales frecuentes

Tabla 6.4: Reformulación de afirmaciones imprecisas.

Formulación imprecisa	Formulación más precisa
“Python no se compila.”	Una implementación habitual compila a <i>bytecode</i> y lo ejecuta en una máquina virtual.
“C es de bajo nivel.”	Suele describirse informalmente como nivel medio: ofrece abstracciones y acceso cercano a la máquina.
“Con GC no hay fugas.”	Puede existir retención de objetos alcanzables que ya no poseen utilidad.
“La memoria automática libera todos los recursos.”	Archivos, sockets y bloqueos suelen requerir cierre determinista.
“Rust administra memoria manualmente.”	El programador expresa propiedad; el compilador verifica reglas e inserta la destrucción.
“Más abstracción implica menos rendimiento.”	El resultado depende también del algoritmo, compilador, entorno y hardware.
“Orientación a objetos significa usar clases.”	Las clases son una herramienta; el paradigma trata de objetos, responsabilidades e interacción.

6.4 Criterios de selección

La decisión debe comenzar por los requisitos del sistema, no por una preferencia aislada de lenguaje.

Sistemas operativos, controladores y microcontroladores.

C, C++ o Rust, según las restricciones, certificaciones, bibliotecas y herramientas disponibles.

Aplicaciones empresariales y servicios generales.

Java, C#, Go y otros lenguajes con GC reducen gran parte del trabajo manual.

Ciencia de datos y automatización.

Python ofrece recuperación automática y un ecosistema amplio, aunque muchas bibliotecas críticas utilizan implementaciones nativas.

Seguridad de memoria sin trazado global.

Rust ofrece garantías estáticas en su subconjunto seguro.

Ecosistema Apple.

Swift utiliza ARC y exige comprender relaciones fuertes, débiles y no propietarias.

Sistemas con latencia estricta.

Deben evaluarse el asignador, las pausas, la fragmentación, las cascadas de destrucción y las operaciones de recursos; la etiqueta del mecanismo no basta.

La elección final también depende del ecosistema, la mantenibilidad, el rendimiento medido, la interoperabilidad y la experiencia del equipo. La administración de memoria es un criterio importante, no el único.

6.5 Matriz de decisión resumida

Tabla 6.5: Adecuación cualitativa según necesidades generales.

Necesidad	Explícita	Trazado	Referencias	Alcance/propiedad
Control directo del punto de liberación	Alto	Bajo	Medio-alto	Alto
Simplicidad para grafos compartidos	Baja	Alta	Media	Media
Recuperación natural de ciclos	No	Sí	No, sin apoyo	Depende del diseño
Ausencia de trazado global	Sí	No	Sí	Sí
Protección estática frente a errores temporales	Baja	Media	Media	Alta en Rust seguro
Curva de aprendizaje inicial	Media	Baja-media	Media	Media-alta

Los valores son orientativos. Una implementación concreta puede alterar sustancialmente la latencia, el consumo y las garantías observables [5, 16].

Capítulo 7

Prácticas de Consolidación

Las actividades de este capítulo relacionan los conceptos con diagnósticos del compilador y del entorno de ejecución. Deben realizarse en archivos separados y en un entorno de prueba. La primera práctica utiliza el detector descrito y evaluado por Serebryany y colaboradores [14].

7.1 Diagnóstico de errores manuales

Objetivo: reconocer una fuga, una doble liberación y un uso después de liberar.

Procedimiento:

1. Guarde cada ejemplo erróneo del Capítulo 3 en un archivo independiente.
2. Compile y ejecute con los comandos siguientes:

```
cc -fsanitize=address -g ejemplo.c -o ejemplo
./ejemplo
```

3. Identifique el tipo de error, la línea y la traza de llamadas.
4. Explique qué regla de propiedad o tiempo de vida fue violada.

7.2 Reescritura mediante RAII

Objetivo: reemplazar propiedad manual por propiedad explícita en el sistema de tipos. Parta de un programa que utilice `new` y `delete`. Sustituya el puntero propietario por `std::unique_ptr`. Después:

1. elimine la llamada explícita a `delete`;
2. introduzca un retorno temprano y verifique que el destructor se ejecute;
3. intente copiar el `unique_ptr` y analice el diagnóstico;
4. transfiera la propiedad correctamente mediante `std::move`.

7.3 Movimiento y préstamo en Rust

Objetivo: interpretar las garantías del compilador.

Compile un programa que intente usar un `String` después de moverlo. Registre el mensaje del compilador y responda:

1. qué variable era propietaria antes del movimiento;
2. qué variable se convirtió en la nueva propietaria;
3. por qué invalidar el nombre anterior evita una doble liberación;
4. cómo cambiaría el programa si la función recibiera `&str`.

7.4 Ciclos y referencias débiles

Objetivo: distinguir ciclos bajo trazado y conteo de referencias.

Construya un ciclo de dos objetos en Python o Swift. Dibuje el grafo de referencias y determine:

1. cuáles referencias son fuertes;
2. qué ocurre al eliminar las referencias externas;
3. qué mecanismo adicional permite recuperar o romper el ciclo;
4. dónde conviene utilizar una referencia débil.

7.5 Análisis transversal de un lenguaje

Seleccione un lenguaje no utilizado como ejemplo principal en esta guía. Elabore una ficha que incluya:

- nivel de abstracción;
- paradigmas admitidos;
- sistema de tipos;
- cadena de ejecución de una implementación concreta;
- evento que desencadena la recuperación de memoria;
- carácter manual o automático y determinista o no determinista;
- tratamiento de archivos y otros recursos externos.

La ficha debe distinguir propiedades del lenguaje, decisiones de una implementación y características de sus bibliotecas.

7.6 Preguntas de análisis

1. Explique por qué un lenguaje con GC todavía puede presentar crecimiento sostenido de memoria.
2. Diferencie un puntero colgante de un uso después de liberar.
3. Compare la forma en que un ciclo afecta a un recolector por trazado y a un contador puro.
4. Justifique por qué “automático” y “determinista” no deben tratarse como categorías opuestas.
5. Explique por qué el cierre de un archivo es un problema distinto de recuperar la memoria del objeto que lo representa.
6. Proponga una regla de propiedad para una estructura de árbol con referencias a padres e hijos.

7.7 Rúbrica sugerida

Tabla 7.1: Criterios para evaluar las prácticas.

Criterio	Peso	Evidencia esperada
Exactitud conceptual	35 %	Distingue propiedad, alcance, vida, alcanzabilidad y mecanismo.
Análisis del diagnóstico	25 %	Interpreta mensajes y los relaciona con una causa concreta.
Calidad del código	20 %	Ejemplos mínimos, reproducibles y con manejo correcto de errores.
Representación visual	10 %	Grafo o diagrama coherente con las referencias reales.
Claridad de la explicación	10 %	Argumentación precisa y terminología consistente.

Capítulo 8

Conclusiones

Los lenguajes de programación se clasifican simultáneamente por nivel de abstracción, paradigma, sistema de tipos, forma de ejecución, propósito y administración de memoria. Ninguna dimensión basta por sí sola para caracterizar un lenguaje.

La administración de memoria se comprende mejor al identificar el evento que desencadena la recuperación. La orden explícita, la salida del alcance, el contador igual a cero y la pérdida de alcanzabilidad constituyen mecanismos comparables. Manual, automático, determinista e híbrido describen propiedades que pueden superponerse.

Los errores clásicos de la gestión manual -fugas, doble liberación y punteros colgantes- proceden de reglas incorrectas de propiedad y tiempo de vida. Los sistemas automáticos reducen parte de esta carga, pero no conocen la intención semántica ni garantizan el cierre oportuno de recursos externos.

C++ moderno y Rust muestran que es posible automatizar una liberación determinista sin trazado global. El primero utiliza RAII y tipos de propiedad; el segundo integra propiedad y préstamos en las comprobaciones del compilador. El conteo de referencias y los recolectores por trazado ofrecen soluciones diferentes para grafos compartidos.

- La propiedad define la responsabilidad sobre el ciclo de vida.
- Los alias determinan qué accesos continúan existiendo.
- El alcance de un nombre y el tiempo de vida del objeto no son equivalentes.
- La alcanzabilidad no equivale a utilidad.
- Ningún mecanismo sustituye un diseño cuidadoso de datos y recursos.

Referencias

- [1] David F. Bacon, Perry Cheng, and V. T. Rajan. A unified theory of garbage collection. *ACM SIGPLAN Notices*, 39(10):50–68, 2004. doi: 10.1145/1028976.1028982.
- [2] David M. Beazley. *Python Distilled*. Addison-Wesley Professional, 2021. ISBN 978-0-13-417327-6.
- [3] David G. Clarke, John M. Potter, and James Noble. Ownership types for flexible alias protection. In *Proceedings of the 13th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*, pages 48–64. ACM, 1998. doi: 10.1145/286936.286947.
- [4] ISO/IEC. *ISO/IEC 14882:2024: Programming Languages — C++*. International Organization for Standardization, 7 edition, 2024.
- [5] Richard Jones, Antony Hosking, and Eliot Moss. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman and Hall/CRC, 2 edition, 2023. ISBN 978-1-032-23178-5. doi: 10.1201/9781003276142.
- [6] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL):66:1–66:34, 2018. doi: 10.1145/3158154.
- [7] Steve Klabnik and Carol Nichols. *The Rust Programming Language*. No Starch Press, 2 edition, 2023. ISBN 978-1-7185-0310-6.
- [8] Matthew Mathias, John Gallagher, and Mikey Ward. *Swift Programming: The Big Nerd Ranch Guide*. Big Nerd Ranch Guides, 3 edition, 2020. ISBN 978-0-13-526660-1.
- [9] David A. Patterson and John L. Hennessy. *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*. Morgan Kaufmann, 2 edition, 2020. ISBN 978-0-12-820331-6.
- [10] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002. ISBN 978-0-262-16209-8.
- [11] Michael L. Scott. *Programming Language Pragmatics*. Morgan Kaufmann, 4 edition, 2015. ISBN 978-0-12-410409-9.
- [12] Robert C. Seacord. *Effective C: An Introduction to Professional C Programming*. No Starch Press, 2 edition, 2024. ISBN 978-1-7185-0412-7.
- [13] Robert W. Sebesta. *Concepts of Programming Languages*. Pearson, 12 edition, 2022. ISBN 978-1-292-43682-1.
- [14] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. AddressSanitizer: A fast address sanity checker. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pages 309–318. USENIX Association, 2012. ISBN 978-931971-93-5.
- [15] Bjarne Stroustrup. *A Tour of C++*. Addison-Wesley Professional, 3 edition, 2022. ISBN 978-0-13-681648-5.
- [16] Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles. Dynamic storage allocation: A survey and critical review. In *Memory Management: International Workshop IWMM 1995*, volume 986 of *Lecture Notes in Computer Science*, pages 1–116. Springer, 1995. doi: 10.1007/3-540-60368-9_19.